

HESIA CoreLib Open Source Reference EN

****Project**:** HESIA CoreLib

****Version**:** 0.1.0

****Language**:** English

****Scope**:** open source security and inference-control library for embedded systems

****Audience**:** firmware engineers, security engineers, OP-TEE maintainers, AI pipeline engineers, language binding maintainers

1. Purpose

HESIA CoreLib is the open source library that gives the HESIA project a public, auditable engineering surface without exposing the private product runtime.

The goal is comparable in spirit to OpenSSL: provide a small, stable, reusable foundation that other projects can build on. The library is not a clone of OpenSSL and does not implement custom cryptographic primitives. It delegates cryptographic operations to established providers and focuses on integration quality, stable ABI design, failure semantics, maintainability, and embedded AI safety controls.

CoreLib exists so that external engineers can evaluate HESIA's engineering posture through real code:

- stable C ABI
- readable implementation
- provider-based cryptography
- explicit error model
- fail-closed behavior
- watchdog and inference guard primitives
- bindings for C, C++, Python, Rust, Swift, and Ada

2. Public vs. private split

The HESIA product remains partially closed source because it contains deployment-specific firmware, operational hardening, OP-TEE flows, model integration, and security-sensitive customer code.

The open source library is deliberately scoped to reusable primitives:

- common error handling
- secure zeroization
- random bytes
- AES-256-GCM provider surface
- ML-KEM and ML-DSA provider surface

- watchdog state machine
- inference request validation
- cross-language ABI

The closed product can use CoreLib internally, but CoreLib must stay useful on its own.

3. Design principles

3.1 Stable ABI first

The primary interface is C. Every other language binding calls the C ABI. This keeps maintenance cost low and avoids divergent behavior between Python, Rust, Swift, C++, and Ada.

3.2 No custom cryptography

CoreLib does not invent symmetric encryption, signature schemes, key exchange algorithms, or KDFs. The public API can expose post-quantum operations, but implementation is delegated to providers such as OpenSSL and liboqs.

3.3 Fail closed

If a provider is not compiled in, the API does not silently downgrade. It returns `HESIA_STATUS_UNSUPPORTED`. Production software must treat unsupported mandatory features as deployment failures.

3.4 Maintenance over cleverness

The code favors flat APIs, explicit sizes, caller-owned buffers, and status codes. That makes it easier to bind from multiple languages and easier to review in security contexts.

3.5 Embedded AI is a security boundary

Inference inputs, outputs, deadlines, and model identity are treated as policy-controlled data. CoreLib does not run a model. It validates the contract around model execution.

4. Repository layout

CoreLib lives in:

```
open_source/hesia-core/
```

Important files:

```
include/hesia/core.h  
include/hesia/core.hpp  
src/hesia_core.c
```

```
Stable C ABI  
Header-only C++ wrapper  
Portable C implementation
```

tests/test_core.c	Unit tests
bindings/python/hesia_core.py	Python ctypes binding
bindings/rust/	Rust FFI crate
bindings/swift/	Swift Package surface
bindings/ada/hesia_core.ads	Ada binding spec
examples/c/quickstart.c	C quickstart
examples/python/quickstart.py	Python quickstart

Project-level documentation:

```
docs/HESIA_CORELIB_EN.md
docs/HESIA_CORELIB_FR.md
docs/HESIA_CORELIB_EN.pdf
docs/HESIA_CORELIB_FR.pdf
```

5. Feature matrix

Feature	API status	Provider	Default behavior
Error model	Implemented	Built-in	Available
Secure zeroization	Implemented	Built-in	Available
OS random	Implemented	OS / OpenSSL	Available when OS provider exists
AES-256-GCM	Implemented API	OpenSSL	UNSUPPORTED without OpenSSL
ML-KEM	Implemented API	liboqs	UNSUPPORTED without liboqs
ML-DSA	Implemented API	liboqs	UNSUPPORTED without liboqs
Watchdog	Implemented	Built-in	Available
Inference guard	Implemented	Built-in	Available
C++ binding	Implemented	ABI wrapper	Available
Python binding	Implemented	ctypes	Available with shared library
Rust binding	Implemented	FFI	Available with linked library
Swift binding	Implemented	Swift Package	Available with linked library
Ada binding	Implemented	GNAT / C ABI	Available with linked library

6. C ABI overview

The C ABI is declared in:

```
open_source/hesia-core/include/hesia/core.h
```

The ABI uses:

- fixed integer types from `stdint.h`
- `size_t` for buffer lengths
- caller-owned buffers
- explicit output lengths
- `hesia_status_t` for all fallible operations
- optional `hesia_error_t` for human-readable diagnostics

Example:

```
hesia_error_t error;
uint8_t out[32];

hesia_error_init(&error);
if (hesia_random(out, sizeof(out), &error) != HESIA_STATUS_OK) {
    fprintf(stderr, "random failed: %s\n", hesia_error_message(&error));
}
```

7. Error model

CoreLib does not throw exceptions from the C ABI. All fallible calls return a `hesia_status_t`.

Important statuses:

- `HESIA_STATUS_OK`
- `HESIA_STATUS_INVALID_ARGUMENT`
- `HESIA_STATUS_BUFFER_TOO_SMALL`
- `HESIA_STATUS_UNSUPPORTED`
- `HESIA_STATUS_CRYPTO_ERROR`
- `HESIA_STATUS_AUTHENTICATION_FAILED`
- `HESIA_STATUS_POLICY_DENIED`
- `HESIA_STATUS_WATCHDOG_EXPIRED`
- `HESIA_STATUS_INFERENCE_REJECTED`
- `HESIA_STATUS_IO_ERROR`
- `HESIA_STATUS_INTERNAL_ERROR`

Rule for production code:

If a feature is mandatory and CoreLib returns `HESIA_STATUS_UNSUPPORTED`, abort startup.

8. Cryptography model

8.1 Symmetric encryption

The AEAD API currently exposes:

- HESIA_AEAD_AES_256_GCM

The implementation is compiled only when OpenSSL support is enabled. Without OpenSSL, the API returns HESIA_STATUS_UNSUPPORTED.

8.2 Post-quantum algorithms

The PQC API currently exposes:

- HESIA_PQC_ML_KEM_768
- HESIA_PQC_ML_KEM_1024
- HESIA_PQC_ML_DSA_65
- HESIA_PQC_ML_DSA_87

The implementation is compiled only when liboqs support is enabled. Without liboqs, the API returns HESIA_STATUS_UNSUPPORTED.

8.3 Provider rule

CoreLib owns:

- API stability
- size checks
- fail-closed behavior
- language binding consistency

Providers own:

- cryptographic primitive implementation
- algorithm-specific constant-time behavior
- low-level key generation and signing details

9. Watchdog module

The watchdog module is a small state machine for embedded loops and AI pipelines:

1. initialize with timeout and allowed missed deadlines
2. arm
3. heartbeat from the controlled loop
4. check from a supervisor
5. fail with HESIA_STATUS_WATCHDOG_EXPIRED when policy is violated

This is useful for:

- inference loops
- sensor ingestion
- secure channel workers
- OP-TEE maintenance helpers
- pipeline stages where silent hangs are dangerous

10. Inference guard module

CoreLib does not execute neural networks. It validates execution contracts before model execution.

The contract includes:

- model id
- model version
- maximum input size
- maximum output size
- maximum latency policy
- optional signed-model digest requirement

The request includes:

- input pointer and length
- output buffer and capacity
- requested deadline

The guard rejects:

- missing model identity
- unconfigured limits
- oversized inputs
- oversized outputs
- deadlines outside policy
- missing digest when signed model policy is required

This keeps AI pipelines easier to audit because the security boundary sits before the runtime engine.

11. Language bindings

11.1 C

C is the primary ABI and the best target for firmware, OP-TEE host tools, and restricted embedded systems.

11.2 C++

The C++ header wraps status codes in exceptions and provides RAIL-style helpers for watchdog use.

11.3 Python

The Python binding uses ctypes. It is intended for tests, operations tooling, CI probes, and integration scripts.

11.4 Rust

The Rust crate exposes FFI definitions and small safe wrappers for version, feature checks, random bytes, and watchdog control.

11.5 Swift

The Swift Package wraps the C target and provides a small Swift-native error surface.

11.6 Ada

The Ada spec maps the C ABI for GNAT-based systems and safety-oriented codebases.

12. Build recipes

12.1 Minimal portable build

```
cd open_source/hesia-core
cc -Iinclude -c src/hesia_core.c -o hesia_core.o
ar rcs libhesia_core.a hesia_core.o
cc -Iinclude tests/test_core.c libhesia_core.a -o test_core
./test_core
```

On Windows with GCC, link bcrypt for OS random:

```
gcc -Iinclude tests/test_core.c src/hesia_core.c -lbcrypt -o test_core.exe
test_core.exe
```

12.2 CMake build

```
cmake -S open_source/hesia-core -B open_source/hesia-core/build
cmake --build open_source/hesia-core/build
ctest --test-dir open_source/hesia-core/build
```

12.3 Provider build

```
cmake -S open_source/hesia-core -B open_source/hesia-core/build-provider \
-DHESIA_CORE_WITH_OPENSSL=ON \
-DHESIA_CORE_WITH_LIBOQS=ON
cmake --build open_source/hesia-core/build-provider
```

For liboqs, set LIBOQS_ROOT when the provider is not installed in a standard location.

13. OP-TEE integration pattern

CoreLib should not replace the OP-TEE trusted application. The correct split is:

- OP-TEE TA owns sealed secrets, signing keys, rollback metadata, and hardware-bound policy
- CoreLib owns host-side ABI, policy checks, error handling, and language integration

Recommended integration:

1. Use CoreLib errors and status codes in host utilities.
2. Use CoreLib PQC size helpers and verification APIs outside the TA.
3. Keep private ML-DSA signing in the TA or HSM in production.
4. Treat soft-sign or normal-world signing as non-production.
5. Refuse startup when required TA commands are unavailable.

14. Security requirements for release

Before calling a CoreLib release production-ready:

- build and run tests on every target architecture
- pin OpenSSL and liboqs provider versions
- run ABI compatibility checks
- run fuzzing on buffer-handling APIs
- perform negative tests for unsupported provider behavior
- verify that mandatory features fail startup when unavailable
- validate bindings against the same shared library
- publish SBOM and license metadata
- run external review before using it in high-assurance deployments

15. Maintenance model

CoreLib should keep a conservative compatibility policy:

- patch releases may fix behavior without changing ABI

- minor releases may add functions and enum values
- major releases may break ABI, but only with a migration guide
- deprecated APIs should remain for at least one minor cycle

The stable rule:

Do not make language bindings smarter than the C ABI.

16. Roadmap

Near-term:

- CI for Windows, Linux, and ARM64
- provider build validation against the vendored HESIA liboqs tree
- ABI check script
- fuzzing harness for AEAD and inference guard inputs
- pkg-config and CMake package exports
- richer Python wheel packaging

Medium-term:

- OP-TEE host helper adapters
- structured audit events
- signed model manifest verifier
- deterministic test vectors for provider-enabled builds
- integration examples for drone AI pipelines

Long-term:

- external security review
- release signing
- reproducible source archives
- public API stability commitment

17. Validation performed

Validation collected on 2026-05-01:

- Windows local GCC build of tests/test_core.c against src/hesia_core.c: passed.
- Windows local C quickstart build and execution: passed.
- Windows local shared library build loaded through the Python ctypes binding: passed.
- Windows local C++ wrapper compile and execution through include/hesia/core.hpp: passed.

- Windows local Ada spec compilation with GNAT: passed.
- Jetson Orin Nano Super remote C build and execution on ajax@100.101.152.53: passed in /tmp/hesia-core-codex-20260501111412.

This validation covered the provider-free baseline: error handling, secure zeroization, watchdog, inference guard, random provider availability, and fail-closed behavior for unavailable OpenSSL/liboqs operations.

Provider-enabled OpenSSL/liboqs builds still require a dedicated validation pass on the target provider versions before production claims.

18. Positioning

HESIA CoreLib is the open source proof surface for the broader HESIA platform:

- post-quantum native
- maintainable
- language-portable
- embedded-friendly
- OP-TEE compatible
- AI-pipeline aware

Its credibility comes from restraint: it exposes useful primitives, documents limits, delegates cryptography to established providers, and fails closed when the deployment does not match the requested security posture.